Dart Programlama Diline Giriş

Modern, Type-Safe ve Nesne Yönelimli Mimariyi Anlamak

Gemini Notebook

Temel Felsefe: Dart Ekosisteminde Her Şey Bir Nesnedir

1 Tam Nesne Yönelim

Sayılar, fonksiyonlar ve hatta null bile birer nesnedir. İstisnasız her şey Object sınıfından türer.

2 Tür Güvenliği

Tür açıklamaları isteğe bağlıdır (tür çıkarımı yapılır), ancak dil statik olarak tür güvenlidir.

3 Gizlilik Kuralı

public, protected, private anahtar kelimeleri yoktur. _ (alt çizgi) ile başlayan her şey kütüphane özeline aittir.

4 Hatalar ve Uyarılar

Uyarılar kodun çalışmasını engellemez, ancak hatalar çalışmayı durdurur.

```dart
// Tür çıkarımı (Type Inference)
var year = 1977;

// Her şey nesnedir
int? number = null;  // null bile nesnedir
```

Gemini Notebook

İlk Temas: Çalıştırılabilir Her Uygulamanın Kalbi

Her uygulamanın çalışmaya başlaması için üst düzey bir **main()** fonksiyonuna ihtiyacı vardır.

void dönüş türü, bu fonksiyonun geriye bir değer döndürmediğini belirtir.

print() konsola metin yazdırmak için kullanılır.

```
void main() {
  print('Hello, World!');
}
```


Gemini Notebook

Kod Organizasyonu: Yorumlar ve Kütüphane İmportları

Sütun 1: Yorumlar

Standart C-tarzı yorumlar ve IDE/dartdoc entegreli dokümantasyon yorumları.

```
// Tek satırlı yorum.

/*
  Çok satırlı
  yorum bloğu.
*/

/// Dokümantasyon yorumu.
/// IDE ve dartdoc bunu okur.
/// Sizi [Sınıf] ile bağlantıya geçirir.
void feed(Food food) {}
```

Sütun 2: İmportlar (İçe Aktarmalar)

Çekirdek kütüphanelere, harici paketlere veya yerel dosyalara erişim. (Önek, show/hide ve deferred ile tembel yükleme de desteklenir).

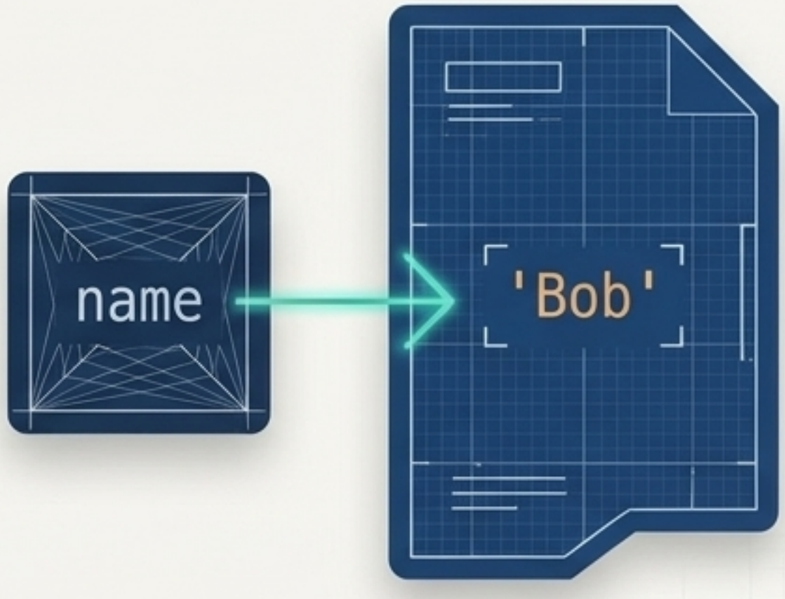
```
import 'dart:math';
import 'package:test/test.dart';
import 'path/to/my_other_file.dart';
```

Gemini Notebook

Değişken Beyanı Matrisi: Tür Çıkarımı ve Dinamiklik

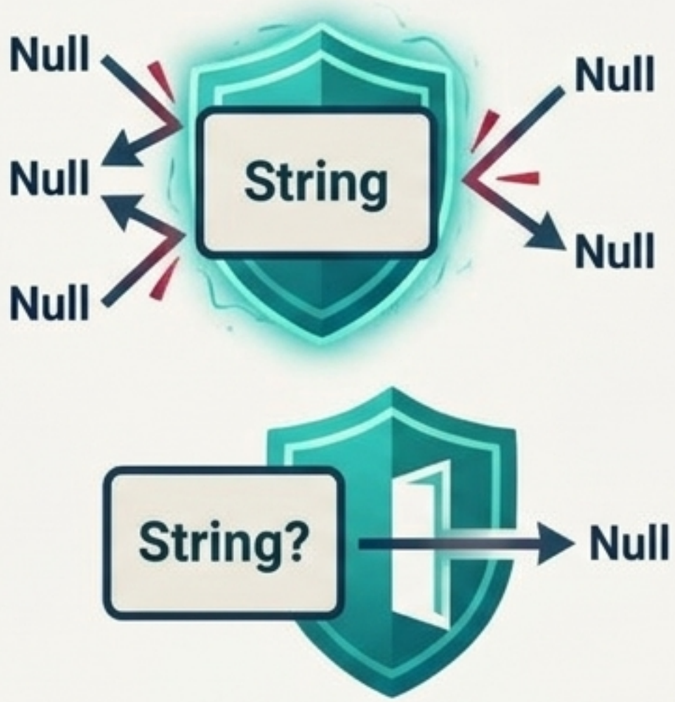
Değişkenler sadece referans saklar. Dart type-safe bir dil olduğu için beyan şekli derleyici davranışını değiştirir.

Beyan Şekli	Tür Belirleme	Derleme Zamanı Kontrolü	Yeniden Atama	Örnek Kod
`var`	İlk değerden çıkarılır	Katı (Tür kilitlenir)	İzin Verilmez	`var name = 'Bob';`
`String`	Açıkça belirtilir	Katı	İzin Verilmez	`String name = 'Bob';`
`Object`	Temel nesne türü	Esnek	İzin Verilir	`Object name = 'Bob';`
`dynamic`	Tür denetimi devre dışı	Yok	İzin Verilir	`dynamic name = 'Bob';`



Gemini Notebook

Null Güvenliği (Null Safety): Kodunuzu Derleme Zamanında Koruyun




```dart
String name = 'Bob'; // Non-nullable (Kalkan devrede)
String? maybeName;   // Nullable (Null izni var)

int? lineCount;
assert(lineCount == null); // Başlatılmamış nullable n

int count;
count = 0; // Kullanmadan önce başlatma zorunludur
```

Sound null safety, null dereferans hatalarını derleme sırasında yakalar. Non-nullable bir yerel değişken kullanılmadan önce mutlaka atanmalıdır.

Gemini Notebook

Değişmezlik Matrisi: final ve const Karşılaştırması

final (Çalışma Zamanı Tek Atama)

- Sadece bir kez atanabilir. Değeri çalışma zamanında (run-time) hesaplanabilir.
- **Örnek Değişkenleri**: Sınıf örnek değişkenleri final olabilir.

```
final name = 'Bob';
final String nickname = 'Bobby';
// name = 'Alice'; // HATA!
```

const (Derleme Zamanı Sabiti)

- Örtük olarak final'dır ancak değeri derleme zamanında (compile-time) bilinmelidir.
- **Derin Değişmezlik**: const nesneler ve alanları tamamen değişmezdir.
- **Örnek Değişkenleri**: Sınıf seviyesinde tek başına const olamaz.

```
const bar = 1000000;
const double atm = 1.01325 * bar;
```


Gemini Notebook

DEĞİŞKENLER

Gecikmeli Başlatma ve Joker Değişkenler

late (Geç Değişkenler)

Tembel başlatma (Lazy init).

Başlatmadan sonra atanan, null olamayan değişken.

Eğer başlatılmadan kullanılırsa çalışma zamanı hatası fırlatır.

```
late String description;
late String temperature = readThermometer(); // Lazy
```

Joker Değişken (Wildcard _)

_ adlı değişken, bağlayıcı olmayan bir yer tutucudur (Dart 3.7+). Aynı kapsamda birden fazla _ çakışmaz.

```
var _ = 1;
for (var _ in list) {}
try {...} catch (_) {}
list.where((_) => true);
```


Gemini Notebook

Operatör Öncelik Hiyerarşisi

İki işlenenli operatörde, işlemi soldaki işlenen belirler. Parantezler okunabilirliği artırır.

Zirve (En Yüksek Öncelik): Unary ++, --, (), [], ., ?., !

2. Katman: Aritmetik: *, /, ~/, %, +, -

3. Katman: Kaydırma ve Bitwise: <<, >>, >>>, &, ^, |

4. Katman: İlişkisel ve Tür Testi: >=, >, <=, <, as, is, is!, ==, !=

5. Katman: Mantıksal: && (AND), || (OR)

6. Katman: Koşullu: ??, ?:

Zemin (En Düşük Öncelik): Cascade .., Atama =, *=, Spread ...

```
// Net okunuş
if ((n % i == 0) && (d % i == 0)) {}
```

```
// Okunması daha zor, ancak eşdeğer
if (n % i == 0 && d % i == 0) {}
```


Gemini Notebook

OPERATÖRLER

Temel Operatörler İş Başında

Aritmetik ve Bitwise

```
assert(5 ~/ 2 == 2); // Tam sayı bölme
assert(5 % 2 == 1);  // Kalan (modül)

b = ++a; // Önce artır
assert((0x22 << 4) == 0x220); // Sola kaydı
```

Eşitlik ve Tür Testi

```
assert(2 == 2); // == null durumunu ele alı

if (employee is Person) { // Tür testi
  employee.firstName = 'Bob';
}

(emp as Person).firstName = 'Bob'; // Tür dö
```

Atama ve Mantıksal

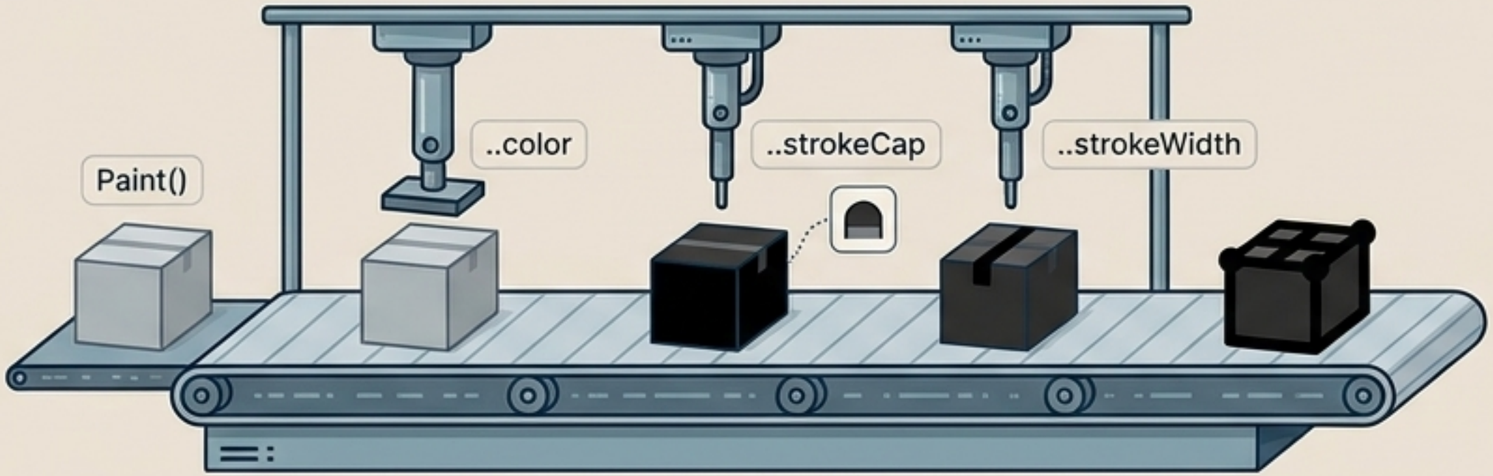
```
b ??= value; // Null ise ata
a *= 3; // Bileşik atama

if (!done && (col == 0 || col == 3)) {}
```


Gemini Notebook

Akıcı Sözdizimi: Basamaklı (Cascade) ve Yayma (Spread) Operatörleri

Basamaklı Gösterim




```
var paint = Paint()
   ..color = Colors.black
   ..strokeCap = StrokeCap.round
   ..strokeWidth = 5.0;
```

Aynı nesne üzerinde işlem zinciri kurar. (Void döndüren çağrıda çalışmaz).

Spread Operatörü

Koleksiyon öğelerini başka bir koleksiyonun içine serpiştirerek ekler.

```
var list = [...a + b];
int x = maybeNull!; // null olmayan doğrulama operatörü (!)
```

Gemini Notebook

Kontrol Akış İfadeleri

- Standart if / else if /else yapıları.
- Koleksiyonlar üzerinde gezinmek için pratik for-in döngüsü.
- Standart for ve while döngüleri.
- Ayrıca desteklenenler: break, continue, switch-case ve hata ayıklama için assert.

```
// İf - Else
if (year >= 2001) {
  print('21st century');
}

// For-in Döngüsü
for (final o in flybyObjects) {
  print(o);
}

// While Döngüsü
while (year < 2016) {
  year += 1;
}
```

Gemini Notebook

Mimari Temeller: Fonksiyonlar ve Kapatmalar (Closures)

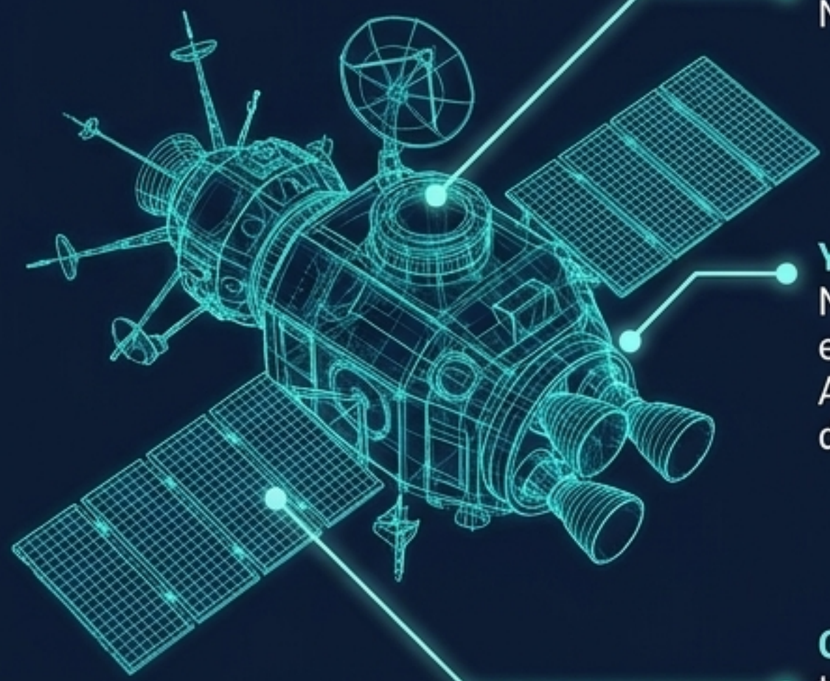
- **Birinci Sınıf Nesneler:** Dart'ta fonksiyonlar da nesnedir. Başka fonksiyonlara argüman olarak iletilebilirler.
- **Ok (Arrow) Sözdizimi:** Tek ifadeli fonksiyonlar için => (ok) kısa yolu okunabilirliği artırır.
- **Tür Belirtme:** Argüman ve dönüş türlerinin belirtilmesi kesinlikle önerilir.

```dart
// Standart Fonksiyon
int fibonacci(int n) {
  if (n == 0 || n == 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

// Ok Sözdizimi ve Yüksek Dereceli Fonksiyonlar
flybyObjects
  .where((n) => n.contains('turn'))
  .forEach(print);
```

Gemini Notebook

Sınıflar (Classes): Nesne Şablonlarını İnşa Etmek



Özellikler (Properties): Nesnenin durumu.

Yapıcılar (Constructors): Nesneyi bellekte var eden metot. Adlandırılmış yapıcılar desteklenir.

Getters: Orijinal veriden türetilen salt salt okunur özellikler (=> kullanılarak hesaplanır).

```
class Spacecraft {
  String name;
  DateTime? launchDate;

  // Getter (Salt Okunur Özellik)
  int? get launchYear => launchDate?.year;

  // Standart Yapıcı
  Spacecraft(this.name, this.launchDate);

  // Adlandırılmış Yapıcı (Named Constructor)
  Spacecraft.unlaunched(String name) : this(name, null);
}
```

Gemini Notebook

Enumlar (Numaralandırmalar): Önceden Tanımlı Sabit Kümeleri

- Sadece basit **etiketler** değil, gelişmiş **enumlardır. Alan, yapıcı, getter ve metot** içerebilirler.
- **Nokta Kısaltması:** Dart'ın tür çıkarımı sayesinde, bağlam netse PlanetType.venus yerine sadece .venus yazılabilir.

```dart
enum PlanetType { terrestrial, gas, ice }

// Tür çıkarımı sayesinde nokta kısaltması
PlanetType myPlanet = .terrestrial;

// Koşullu ifade (Conditional Expression) örneği
String playerName(String? name) => name ?? 'Guest';
```

Gemini Notebook

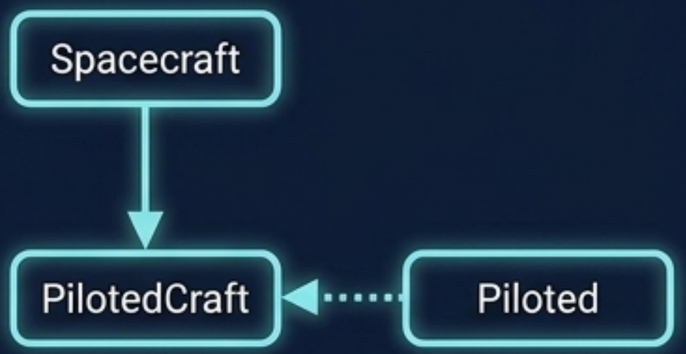
Dart Nesne Yönelim Matrisi: Miras, Karışımlar ve Arayüzler

extends (Miras / Kalıtım)

Davranış: Tek taraflı kalıtım. Bir alt sınıf, üst sınıfın özelliklerini alır ve genişletir.

```
class Orbiter extends Spacecraft {
  double altitude;
}
```

with (Mixin / Karışımlar)



Davranış: Kodu birden fazla hiyerarşide yeniden kullanma. Sınıfa yetenek ekler.

```
mixin Piloted {
  int astronauts = 1;
}

class PilotedCraft extends Spacecraft
  with Piloted {}
```

implements (Arayüzler)

Davranış: Başka bir sınıfın API imzasını kopyalamayı zorunlu kılar, uygulamayı değil.

```
class MockSpaceship
  implements Spacecraft
}
```

Gemini Notebook

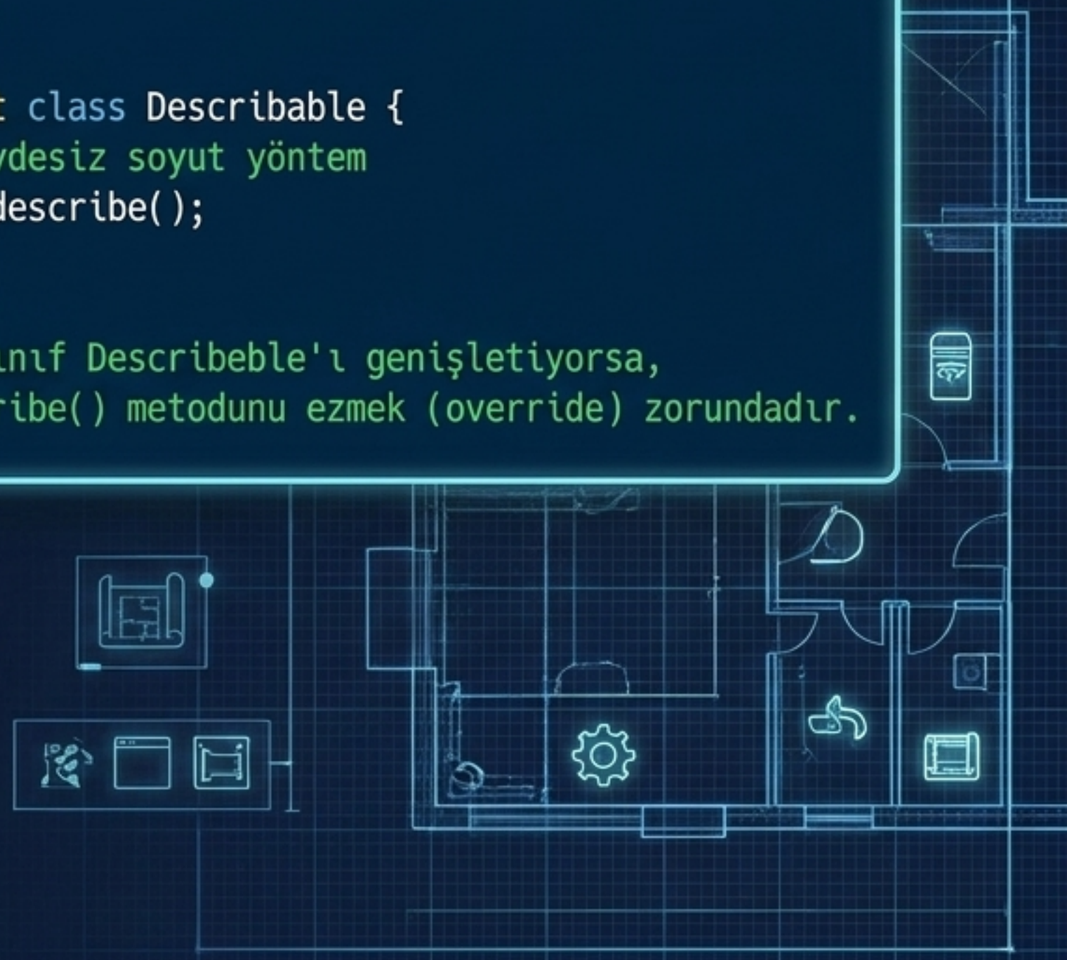
Soyut Sınıflar (Abstract Classes) ve Arayüzler

- **Soyut Sınıf (abstract class):** Doğrudan örneklenemez (instantiate edilemez). Gövdesiz soyut yöntemler (metotlar) içerebilir. Alt sınıflar bu yöntemleri doldurmak zorundadır.

- **Soyut Arayüz (abstract interface):** Örneklenemez ve genişletilemez (extends edilemez), ancak implements ile uygulanabilir. Saf bir API sözleşmesi tanımlamak için mükemmeldir.

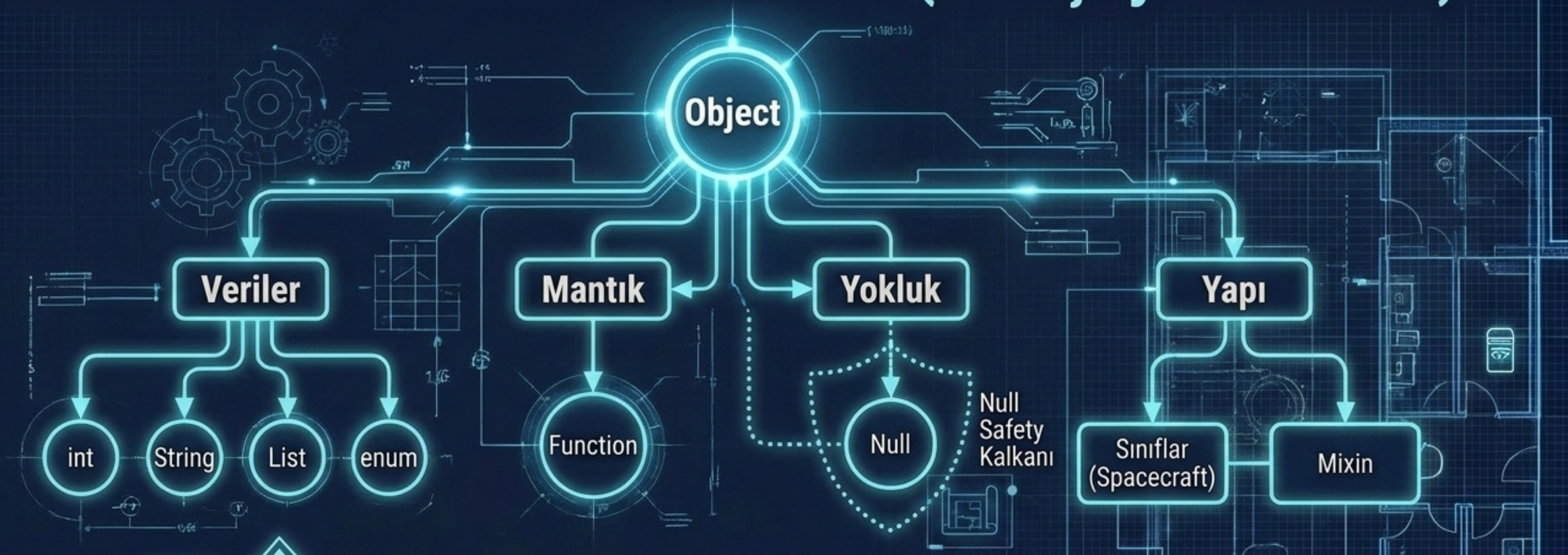
```
abstract class Describable {
  // Gövdesiz soyut yöntem
  void describe();
}

// Bu sınıf Describeble'ı genişletiyorsa,
// describe() metodunu ezmek (override) zorundadır.
```



Gemini Notebook

Sentez: Dart Nesne Ekosistemi (Her Şey Nesnedir)



Dart'ın gücü bu homojen yapıdan gelir. Bir fonksiyonu bir Listeye ekleyebilir, bir **sayıyı bir değişkene atar gibi atayabilir** veya **Null** değerini yönetebilirsiniz; çünkü temel düzeyde her biri **Object** sınıfından türetilen varlıklardır.

Gemini Notebook

Gerçek Dünya Tahmin Edilemez: İstisnalar (Exceptions)

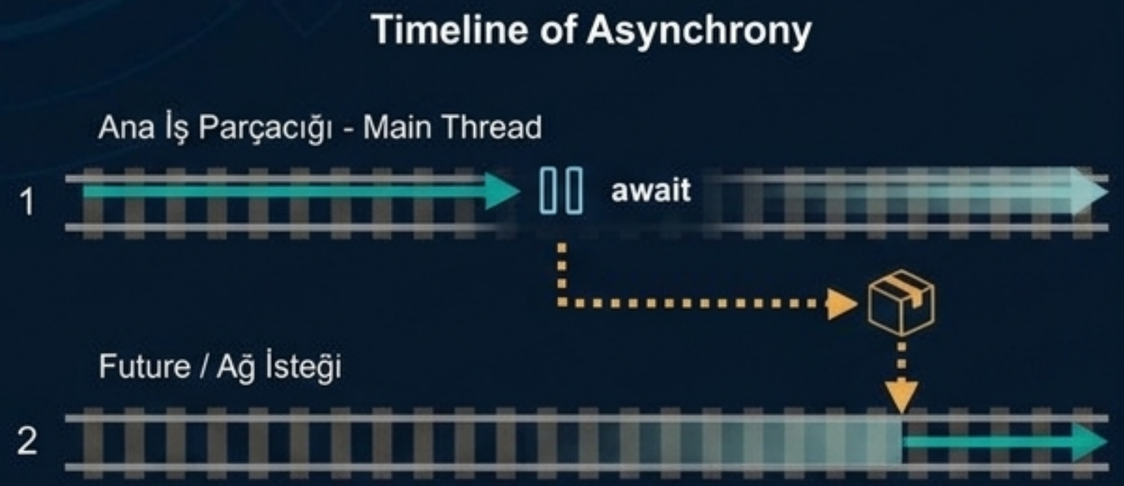
- **throw** ile istisna oluşturulur (Fırlatılır).
- **try / on / catch / finally** blokları ile güvenli ağ (safety net) kurulur ve hata yakalanır.
- Dart'ta **Error** (programcı hataları) ve **Exception** (öngörülebilir çalışma zamanı hataları) farklı kavramlardır.

```dart
throw StateError('No astronauts.');

try {
  // Riskli kodlar...
} on IOException catch (e) {
  // Belirli bir hata türünü yakalama
  print('Could not describe: $e');
} finally {
  // Hata olsa da olmasa da temizlik yap
  flybyObjects.clear();
}
```


Gemini Notebook

Asenkron Programlama: Zamanı Yönetmek

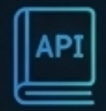


async ve await, callback cehennemini çözer. Asenkron kodu sanki senkronmuş gibi okunaklı hale getirir. **Future** ve **Stream** çekirdek yapılardır.

```
Future<void> printWithDelay(String m) async {
  // İşlem bitene kadar bekle, ancak ana UI'ı dondurma
  await Future.delayed(oneSecond);
  print(m);
}
```

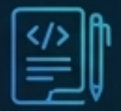

Gemini Notebook

Sonraki Adımlar: Dart Ustalığına Doğru



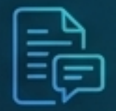
Çekirdek API

Temel kütüphane belgeleri ve Dart API referansı, günlük geliştirme sürecinizdeki en büyük yardımcınızdır.



Stil Kılavuzu (Style Guide)

Yazılan kodların Dart stil kılavuzu kurallarına (Effective Dart) uygunluğunu inceleyin. Dilin idiomlarını (deyimlerini) öğrenmek sizi hızlandırır.



Dokümantasyon Yorumları

dart doc aracını kullanarak HTML dokümantasyonunuzu üretmeyi alışkanlık haline getirin.

Dart'ın **type-safe** ve nesne yönelimli mimarisi, ölçeklenebilir ve sağlam uygulamalar inşa etmeniz için tasarlanmıştır.

Gemini Notebook